Lab W1

Navigating the Linux Command Line: Foundations for Java Development

Note. For many of you, this is a new experience, and you are likely to hit a few snags that you will need to work through on your own. That is by design. Learning to troubleshoot takes time, research, and a willingness to sit with a problem instead of walking away from it. I am happy to help, but my availability is limited. I am not able to help after 6 pm or on weekends, so start this lab early in the week and leave yourself time to stop by during office hours if you get stuck. Most troubleshooting cannot be solved over email, since I need to actually see what is happening on your machine. Plan to visit in person or reach me on Teams so I can see the issue directly.

Why This Matters

In this course, software development is not an abstract exercise confined to an editor window. The Java programs you write are compiled, run, and often deployed on Linux systems, where they read configuration files, write logs, and interact directly with the operating system. Most of the time you will write, compile, and run your programs inside an IDE, but you should still be comfortable navigating a Linux system from the command line, since you may need to compile and run a program directly from the command line from time to time. You were already introduced to some of this when setting up your environment.

This lesson introduces the Linux command line as a tool for navigation and observation in more detail, and finishes by having you compile and run your first Java program without an IDE. To begin, log in to your Xubuntu VM (or your own Linux install; it is unclear how much of this applies directly to macOS) and open a terminal.

Learning Goals

By the end of this lesson, you will be able to:

- Navigate the Linux file system confidently
- Understand where important system and software files are located
- View files and directories without modifying them
- Recognize how Linux permissions affect what you can see and do
- Use help and documentation tools to learn commands independently
- Compile and run a simple Java program directly from the terminal

About Your Screenshots

Throughout this lab, you will be asked to take a screenshot to show that you completed a part of the assignment. A few things apply to all of them:

- Take one screenshot per part of the assignment. There are seven in total, covering Sections 2 through 8.
- A screenshot does not need to show everything you did for that part. Get as much of the relevant terminal output on screen as you reasonably can, and do not worry if some of it scrolled out of view.
- Each image must be an actual screenshot taken on your computer, not a photo taken with a phone or camera. If you do not already know how to take a screenshot on your system,

look this up, for example a keyboard shortcut or a built in screenshot tool.

- **File naming:** name each screenshot to match the section it documents, labW1_2.png for Section 2, labW1_3.png for Section 3, and so on through labW1_8.png

1. What Is the Command Line?

The command line is a text based interface that allows you to interact directly with the operating system. Instead of clicking through menus, you type commands that tell the system exactly what to do. Most software build, deployment, and administration tasks rely on the command line because it is:

- Precise
- Scriptable
- Available on all servers

As a Java developer, you will use the command line mainly to observe and inspect your system. From time to time, you may also use it to compile a source file with javac and run it with java, though most of your day to day compiling and running will happen inside an IDE. Open a terminal, either through the menu system or the shortcut key Ctrl+Alt+T.

2. Your Location in the File System - The Idea of "Where You Are"

In Linux, you are always "inside" a directory. Commands operate relative to your current location.

Essential Commands

- `pwd`: check your current directory
- `ls`: list files in the current directory
- `cd directory_name`: move into another directory
- `cd ..`: move up one level
- `cd ~`: move to your home directory

Practice these commands and take a screenshot of the results. Save it as labW1_2.png.

1. Run pwd
2. Run ls
3. Move into a subdirectory using cd
4. Return to your home directory

3. Understanding the Linux Directory Structure

Linux organizes files in a hierarchy (like folders in Windows), starting at the root directory /.

Table 1. *Key Linux directories used in this course*

Directory	What It Holds
/	Root of the file system
/home	User home directories
/etc	Configuration files

Directory	What It Holds
/var	Variable data (logs, and more)
/var/lib	Application data (including databases)
/var/log	Log files
/usr/bin	User commands

Practice by navigating the following directories and listing their contents. Take a screenshot of the results and save it as labW1_3.png. Do not worry if you do not yet understand everything you see.

1. `cd /etc`
2. `ls`
3. `cd /var`
4. `ls`
5. `cd /var/log`
6. `ls`

Curious? Linux exposes live information about your hardware and the running system as ordinary text files inside /proc, even though nothing is actually stored there on disk. Try cat /proc/cpuinfo or cat /proc/meminfo and see what shows up. Nothing you view there can break your system, so it is safe to explore. If you want to go further, try ps aux to see every process currently running, or cat /proc/version to see exactly which kernel you are running. This is optional and not required for submission, but it is a good rabbit hole if you want one.

4. Viewing Files Without Editing Them

When working with software, reading files is safe. Editing files requires care. Some common commands are cat, less, more, tail, head, and grep.

Table 2. *Commands for viewing file contents*

Command	What It Does	When to Use It	Example
cat	Displays the entire contents of a file at once	Small files or quick checks	cat file.txt
less	View a file one screen at a time (scrollable)	Large files, logs, safe viewing	less file.txt
more	View a file page by page (forward only)	Simple paging, limited navigation	more file.txt
tail	Shows the last part of a file	Watching logs, recent activity	tail file.txt
head	Shows the first part of a file	Checking headers or formats	head file.txt
grep	Searches for text patterns inside files	Finding errors, keywords, and entries	grep "error" file.txt

Practice these by looking at your bash history file (this keeps a history of all the commands you have executed at the command line). Take a screenshot of the results and save it as labW1_4.png.

1. `cd ~`
2. `ls -al`
3. `cat .bash_history`
4. `less .bash_history`
5. `head .bash_history`
6. `tail .bash_history`
7. `grep "cd" .bash_history`

5. File Ownership and Permissions (Introduction)

Linux controls access using owner, group, and permission bits. You saw that information when you ran ls -al. The 'a' listed all the files, and the 'l' listed the permissions. Note: you do not need to be in a directory to list its contents.

This tells you:

- Who owns the file
- Which group it belongs to
- Who can read, write, or execute it

Practice by listing the permissions of the files in /home, /var, and /var/log. Take a screenshot of the results and save it as labW1_5.png. You can also use -al for this part.

1. `ls -l /home`
1. `ls -al /home`
1. `ls -alh /home`
2. `ls -l /var`
3. `ls -l /var/log`

6. Finding Things on the System

Large systems require search tools. The commands find (search for files) and grep (search inside files) are essential. Use these carefully to narrow searches, or you could end up seeing a lot of files.

Practice by finding all the log files in /var and which files in /etc mention "network." Take a screenshot of the results and save it as labW1_6.png.

1. `find /var -name "*log*"`
2. `grep -R "network" /etc`

7. Getting Help

Linux developers take great care to create man pages (manual pages), so you do not have to remember everything. These pages often give a lot of information about a command, its options, and examples. Administrators can tell stories all day about how they solved an issue just by reading the man pages. Many tools also have a short built in help option.

Practice using the help system. Take a screenshot of the results and save it as labW1_7.png.

1. `ls --help`
2. `man git`
3. `man -k "commit" git`

8. Compiling and Running a Java Program

You will mostly compile and run Java from within an IDE in this course, which does this for you with a single button click. That is convenient, but it hides what is actually happening, and there will be times, whether debugging an odd IDE issue or working on a remote server, when you need to do it directly. This section walks through the same process by hand, using only the command line and the JDK tools you already have installed.

Create a Working Directory

Navigate to (or create) a directory to hold this lab's work, then move into it.

1. `mkdir -p ~/cis3300/labW1`
2. `cd ~/cis3300/labW1`

Create the Source File

Open a plain text editor from the terminal and create a new file named HelloWorld.java. You may use either nano or vim, whichever you prefer. We will use vim often during lecture, so this is a good chance to practice it if you have not already.

Option A: nano

1. `nano HelloWorld.java`

Type the code below exactly as written, including capitalization. Java is case sensitive, and the file name must match the class name.

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Save the file and exit the editor. In nano, this is Ctrl+O to save, Enter to confirm the file name, then Ctrl+X to exit.

Option B: vim

1. `vim HelloWorld.java`

Press i to enter insert mode, then type the same code shown above exactly as written.

When you are done, press Esc to leave insert mode. Then type :wq and press Enter to save and quit.

Compile and Run

The javac command compiles your source file into bytecode, and the java command runs it.

1. `javac HelloWorld.java`
2. `ls`
3. `java HelloWorld`

Note. Step 2 is there so you can see that compiling created a new file, HelloWorld.class. That is the compiled bytecode. The java command runs the compiled class, not the .java source file, which is why you type java HelloWorld and not java HelloWorld.java.

Confirm that the terminal prints "Hello, World!". Take a screenshot showing as much of the compile command, the directory listing, and the program's output as you can fit on screen, and save it as labW1_8.png.

Submission

Create a folder named labw01 inside your cis3300 repository (that is, cis3300/labw01). Place all screenshots from this lab, along with your HelloWorld.java file, in that folder. Commit and push the folder to your repository. Return to Lab W1 in Moodle and put "done" in the text submission box. Do not upload images to the submission box.